

CSE 221: Algorithms

Chapter 8 — Greedy Algorithms

Prepared by Ariyan Hossain [AYO]

A **Greedy Algorithm** builds a solution step by step by always making the **best possible local choice** based on the information currently available.

Key idea

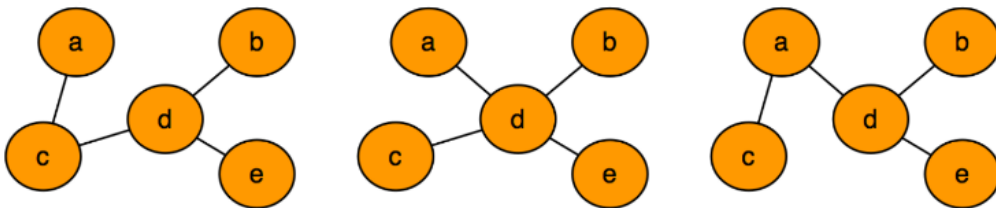
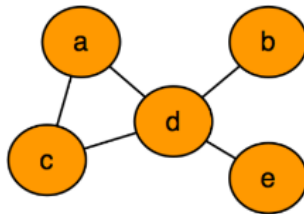
At each step, pick the choice that looks best *right now*. This often leads to efficient solutions to optimization problems.

1 Spanning Tree and MST

A **spanning tree** of an undirected graph $G = (V, E)$ is a subgraph T of G such that:

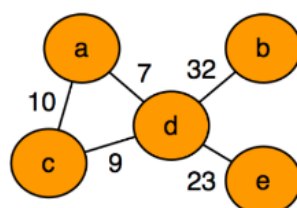
- T is a **tree** (connected, acyclic, and has $|V| - 1$ edges)
- T contains **all vertices** of G

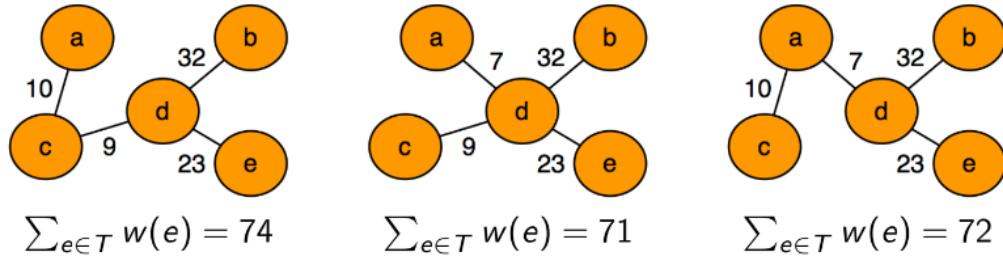
Given the following **unweighted** graph:



All possible unweighted spanning trees from the given graph

Given the following **weighted** graph:





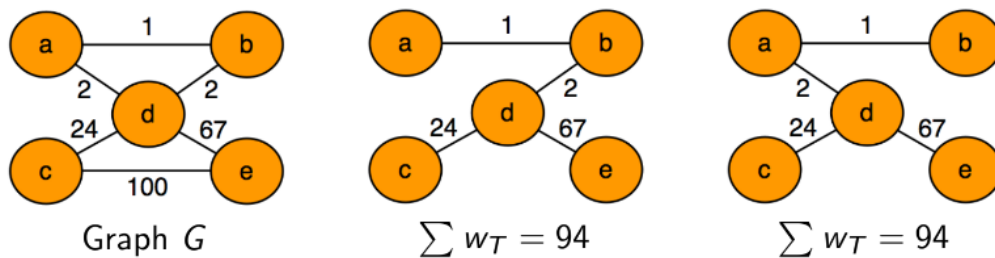
All possible weighted spanning trees from the given graph

A **Minimum Spanning Tree (MST)** is a spanning tree whose total edge weight is minimum among all spanning trees.



Generation of a minimum spanning tree by removing the edge from a to c

MSTs may not be unique.



Generation of multiple minimum spanning trees by removing different edges

2 Kruskal's Algorithm

Kruskal's algorithm is a greedy algorithm for finding the **minimum spanning tree** of a weighted undirected graph. The greedy choice is to pick the **smallest-weight edge that does not cause a cycle** in the MST constructed so far.

2.1 Disjoint Set / Union-Find Data Structure

It stores a non-overlapping or disjoint subset of elements, which is used to avoid cycles when creating MSTs using Kruskal's Algorithm. The main operations are:

- **MAKE-SET(u)**: Create a new set containing a single element u .
- **FIND-SET(u)**: Determine the subset containing the element u . This can determine if two elements are in the same subset.
- **UNION(u, v)**: Join two subsets containing u and v into a single subset. Here first we have to check if the two subsets belong to the same set. If not, then we cannot perform union.

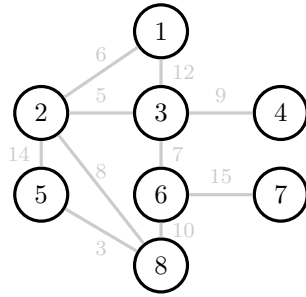
2.2 Simulation (Kruskal's Algorithm)

Step 1:

$w(u, v)$	(u, v)
3	(5,8)
5	(2,3)
6	(1,2)
7	(3,6)
8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)

$|V| = 8$ $|E| = 10$ $|T| = 0$

Vertex sets: $\{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}, \{7\}, \{8\}$

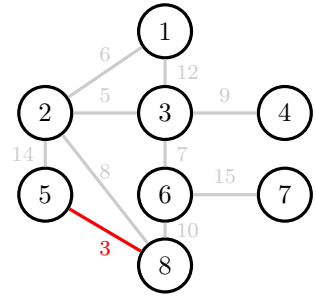


Step 2:

$w(u, v)$	(u, v)
3	(5,8)
5	(2,3)
6	(1,2)
7	(3,6)
8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)

$|V| = 8$ $|E| = 10$ $|T| = 1$

Vertex sets: $\{1\}, \{2\}, \{3\}, \{4\}, \{5, 8\}, \{6\}, \{7\}$

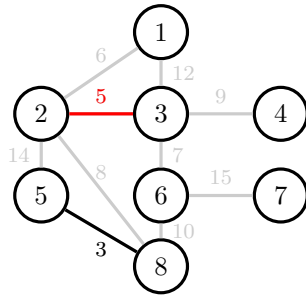


Step 3:

$w(u, v)$	(u, v)
✓ 3	(5,8)
5	(2,3)
6	(1,2)
7	(3,6)
8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)

$|V| = 8$ $|E| = 10$ $|T| = 2$

Vertex sets: $\{1\}, \{2, 3\}, \{4\}, \{5, 8\}, \{6\}, \{7\}$

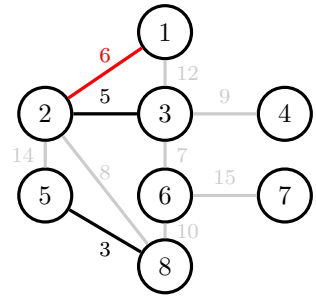


Step 4:

$w(u, v)$	(u, v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
7	(3,6)
8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)

$|V| = 8$ $|E| = 10$ $|T| = 3$

Vertex sets: $\{1, 2, 3\}, \{4\}, \{5, 8\}, \{6\}, \{7\}$

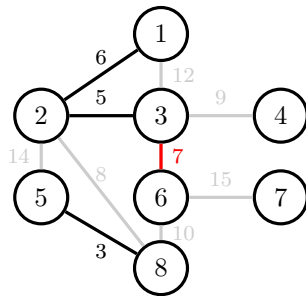


Step 5:

$w(u, v)$	(u, v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
7	(3,6)
8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)

$|V| = 8$ $|E| = 10$ $|T| = 4$

Vertex sets: $\{1, 2, 3, 6\}, \{4\}, \{5, 8\}, \{7\}$

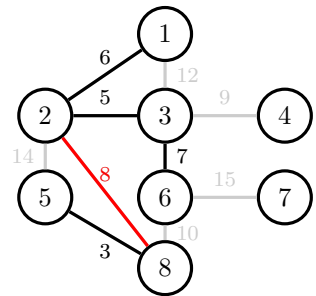


Step 6:

$w(u, v)$	(u, v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
✓ 7	(3,6)
8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)

$|V| = 8$ $|E| = 10$ $|T| = 5$

Vertex sets: $\{1, 2, 3, 5, 6, 8\}, \{4\}, \{7\}$

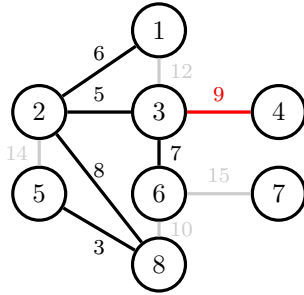


Step 7:

$w(u, v)$	(u, v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
✓ 7	(3,6)
✓ 8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)

$|V| = 8$ $|E| = 10$ $|T| = 6$

Vertex sets: $\{1, 2, 3, 4, 5, 6, 8\}, \{7\}$

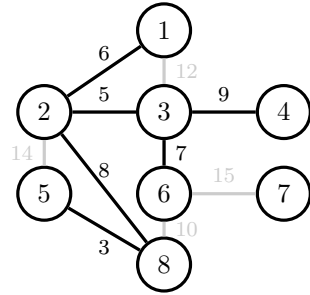


Step 8:

$w(u, v)$	(u, v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
✓ 7	(3,6)
✓ 8	(2,8)
✓ 9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)

$|V| = 8$ $|E| = 10$ $|T| = 6$ (rejected)

Vertex sets: $\{1, 2, 3, 4, 5, 6, 8\}, \{7\}$

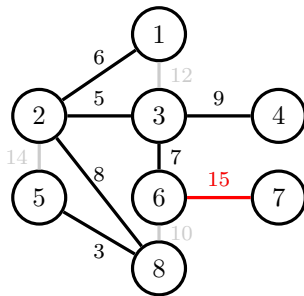


Step 9:

$w(u, v)$	(u, v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
✓ 7	(3,6)
✓ 8	(2,8)
✓ 9	(3,4)
× 10	(6,8)
× 12	(1,3)
× 14	(2,5)
15	(6,7)

$|V| = 8$ $|E| = 10$ $|T| = 7$

Vertex sets: $\{1, 2, 3, 4, 5, 6, 7, 8\}$

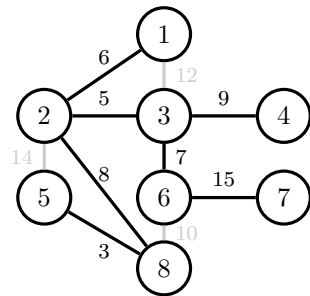


Final State:

$w(u, v)$	(u, v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
✓ 7	(3,6)
✓ 8	(2,8)
✓ 9	(3,4)
× 10	(6,8)
× 12	(1,3)
× 14	(2,5)
✓ 15	(6,7)

$|V| = 8$ $|E| = 10$ $|T| = 7$

Vertex sets: $\{1, 2, 3, 4, 5, 6, 7, 8\}$



2.3 Pseudocode (Kruskal's Algorithm)

Kruskal's Algorithm on graph G

```

KRUSKAL( $G$ )
   $A \leftarrow \{\}$       // empty set to store included edges
  for each  $v$  in  $V[G]$ 
    MAKE-SET( $v$ )
  sort all  $E[G]$  into nondecreasing order by weight
  for each  $e(u, v)$  in  $E[G]$  in sorted order
    if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
      add  $(u, v)$  to  $A$ 
      UNION( $u, v$ )
  return  $A$       // set containing edges of the MST

```

2.4 Time complexity (Kruskal's Algorithm)

To perform Kruskal's Algorithm efficiently, a **disjoint set** (also called **union-find**) data structure is used to keep track of which vertices belong to the same connected component.

- We must first **sort all E edges by weight**, which takes $\Rightarrow \mathcal{O}(E \log E)$
- **MAKE-SET** takes $\mathcal{O}(1)$ per call and is performed once for each vertex $\Rightarrow \mathcal{O}(V)$

- **FIND-SET** takes $\mathcal{O}(\alpha(V))$ per call, where $\alpha(V)$ is the inverse Ackermann function (extremely small and can be treated as nearly constant). Since each edge may require checking both endpoints, it is performed at most $2E$ times $\Rightarrow \mathcal{O}(E\alpha(V))$
- **UNION** takes $\mathcal{O}(\alpha(V))$ per call and is performed at most E times $\Rightarrow \mathcal{O}(E\alpha(V))$

Therefore, Kruskal's algorithm runs in

$$\mathcal{O}(E \log E) + \mathcal{O}(V) + \mathcal{O}(E\alpha(V)) + \mathcal{O}(E\alpha(V)) = \mathcal{O}(E \log E).$$

3 Huffman Coding

Huffman Coding is a greedy algorithm used for **lossless data compression**.

Idea of Huffman

- **Problem:** Not all characters occur with the same frequency. Yet all characters are allocated the same amount of space. According to ASCII, 1 character = 1 byte or 8 bits.
- **Solution:** The idea is to assign variable-length codes to the characters, lengths of the assigned codes are based on the frequencies of corresponding characters. More frequently used characters have shorter codes. By reducing the length of codes for the characters that appear most often, the overall size of the encoded data is minimized.

3.1 Simulation (Huffman Coding)

Let the input be a sentence:

Eerie eyes seen near lake.

Each character occupies **8 bits**. There are a total of **26 characters**. Thus, a total of $8 \times 26 = 208$ **bits** is required. We can compress the string to a smaller size using the Huffman Coding technique.

Huffman coding first creates a tree using the frequencies of the characters and then generates code for each character.

3.1.1 Frequency Table

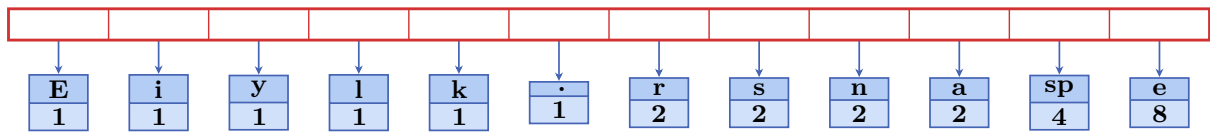
Character	Frequency	Character	Frequency
E	1	s	2
e	8	n	2
r	2	a	2
i	1	l	1
space	4	k	1
y	1	.	1

3.1.2 Prefix Code

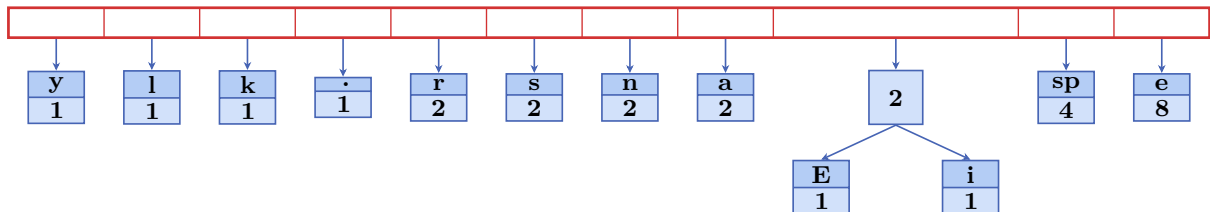
- A prefix code is a code in which no codeword is a prefix of another, ensuring unambiguous decoding (e.g., **01** cannot be a prefix of **01101**).
- Using a binary tree with all characters placed at the leaf nodes guarantees a prefix code, because no leaf (codeword) can lie on the path to another leaf.

3.1.3 Huffman Tree Creation

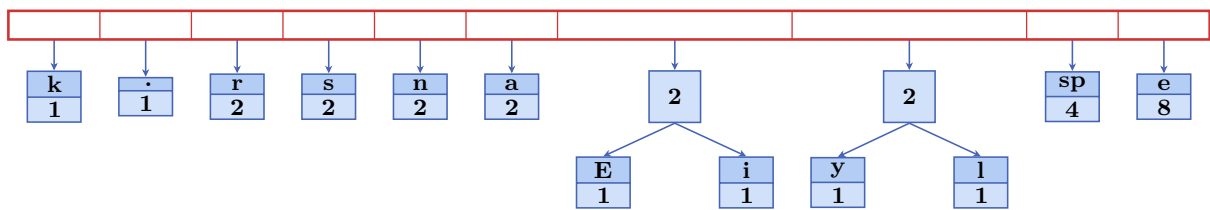
Step 1



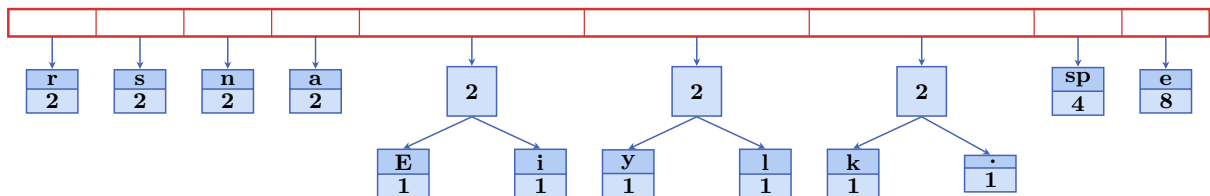
Step 2



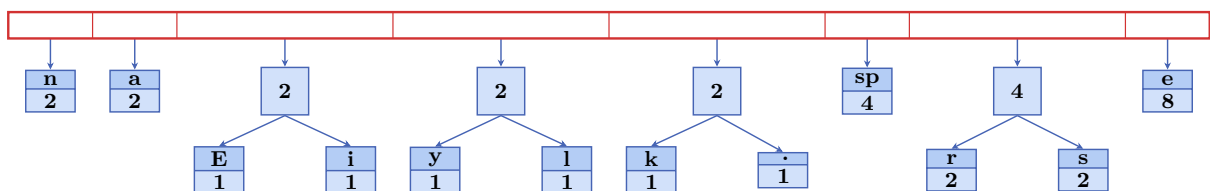
Step 3



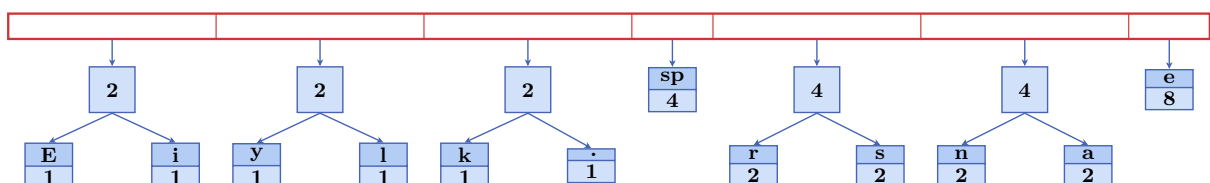
Step 4



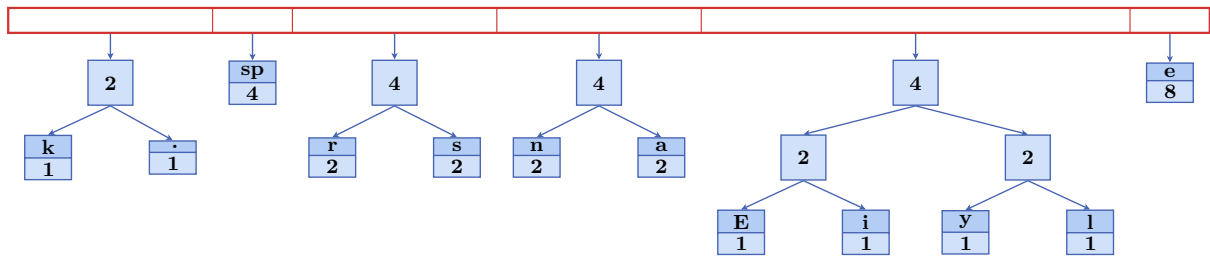
Step 5



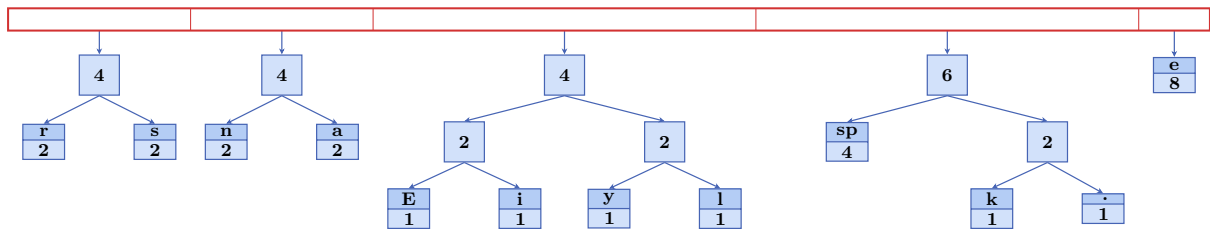
Step 6



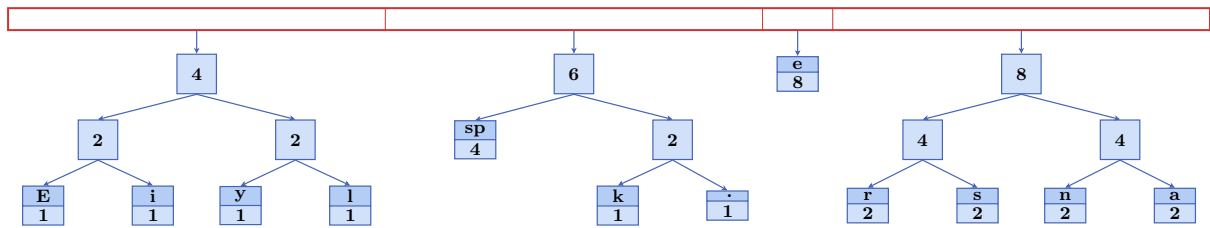
Step 7



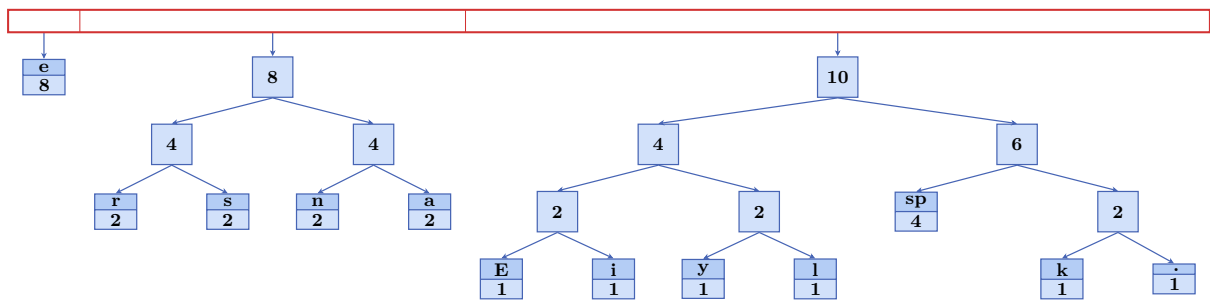
Step 8



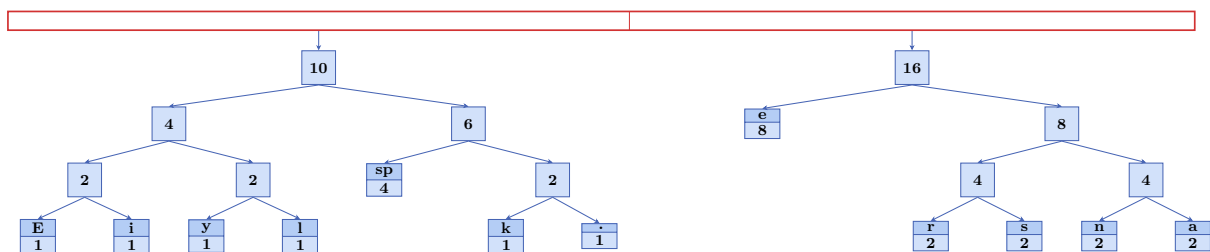
Step 9



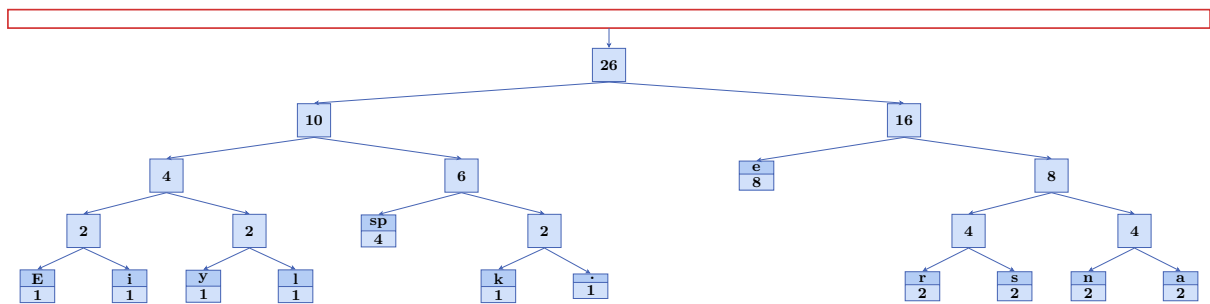
Step 10



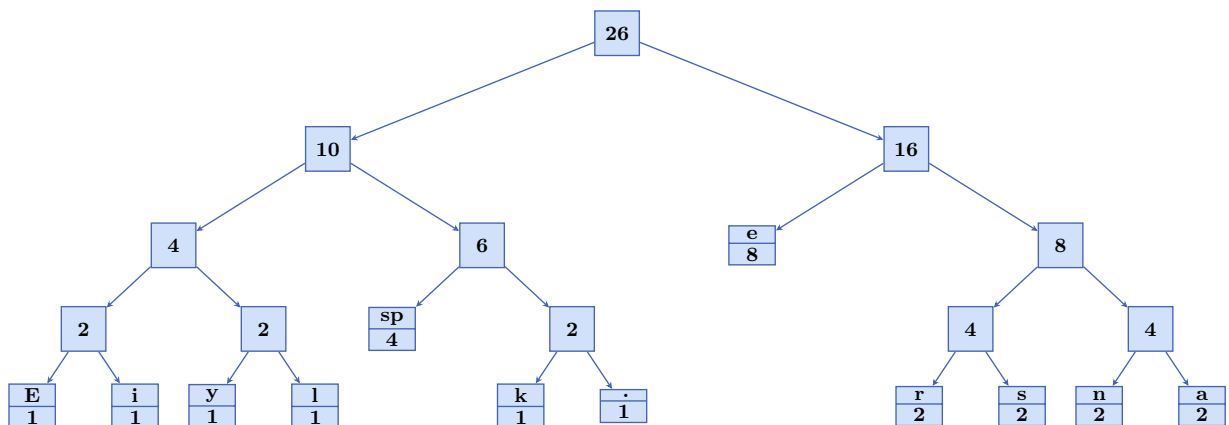
Step 11



Step 12



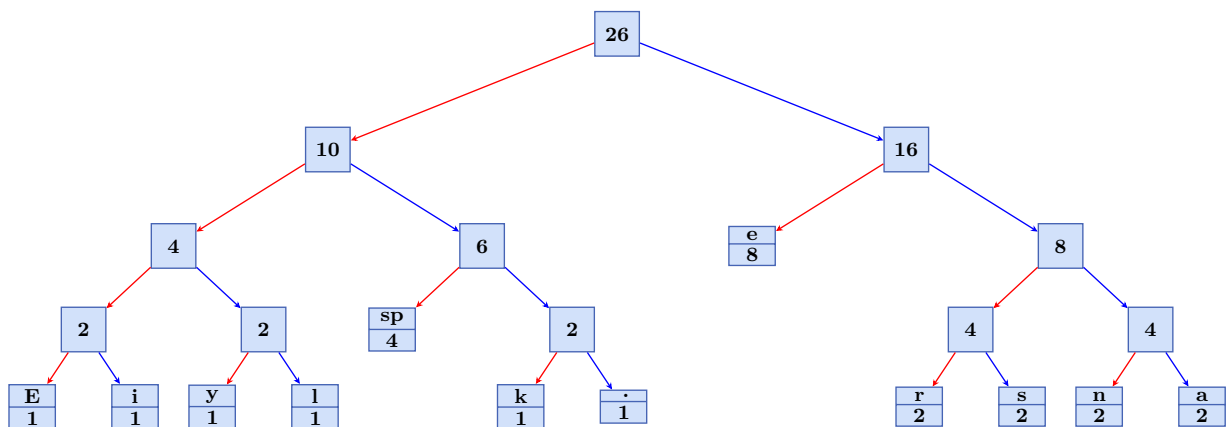
Final Huffman Tree



3.1.4 Huffman Encoding

Encoding Rules (*default*):

- Going Left = 0
- Going Right = 1
- Code word is only completed when a leaf node is reached



Red (left) means 0 and Blue (right) means 1

Char	Freq	Codeword	Bits
e	8	10	2
E	1	0000	4
i	1	0001	4
y	1	0010	4
l	1	0011	4
sp	4	010	3
k	1	0110	4
.	1	0111	4
r	2	1100	4
s	2	1101	4
n	2	1110	4
a	2	1111	4

Huffman Encoding Table

Text: Eerie eyes seen near lake.

Encoded: 000010110000011001010001010110101011010101110101110101111110001000111111011010111

Original Total Bits: $8 \times 26 = 208$ bits

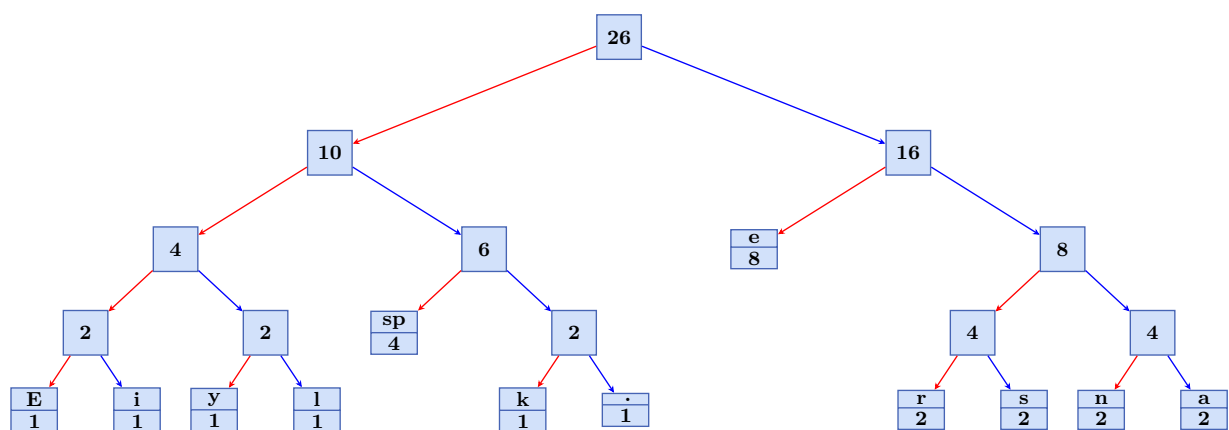
Encoded Bits: 84 bits

Savings: $208 - 84 = 124$ bits ($\approx 60\%$ reduction)

3.1.5 Huffman Decoding

Decoding Rules (*default*):

- Scan the the incoming bit stream with the Huffman Tree
- 0 = Going Left
- 1 = Going Right
- A character is recorded when a leaf node is reached



Red (left) means 0 and Blue (right) means 1

Encoded: 000010110000011001010001010110101011010101110101110101111110001000111111011010111

Decoded Text: Eerie eyes seen near lake.

3.2 Pseudocode (Huffman Tree Construction)

Build Huffman Tree from character set C

```
BUILD-HUFFMAN-TREE( $C$ )
   $Q \leftarrow C$                                 // priority queue containing all characters
  while  $|Q| > 1$ 
    do  $x \leftarrow \text{EXTRACT-MIN}(Q)$ 
        $y \leftarrow \text{EXTRACT-MIN}(Q)$ 
        $z \leftarrow \text{new Node}()$ 
        $z.\text{left} \leftarrow x$ 
        $z.\text{right} \leftarrow y$ 
        $z.\text{freq} \leftarrow x.\text{freq} + y.\text{freq}$ 
        $\text{INSERT}(Q, z)$ 
  return  $\text{EXTRACT-MIN}(Q)$     // return root
```

3.3 Time Complexity (Huffman Coding)

To build a Huffman tree efficiently, a **binary min-heap** is used as the priority queue. Each extraction and insertion operation on the heap takes $\mathcal{O}(\log n)$ time, where n is the number of unique characters.

- **Frequency count** using a linear scan of the input (where $m = \text{input size}$) $\Rightarrow \mathcal{O}(m)$
- **Min-heap creation** swims n characters/nodes into the heap $\Rightarrow \mathcal{O}(n \log n)$
- **Build Huffman tree** performs Extract-Min twice and one insertion, repeated $n - 1$ times $\Rightarrow \mathcal{O}(n \log n)$
- **Generate codes** via DFS assigns a code to each of the n leaf nodes $\Rightarrow \mathcal{O}(n)$

Therefore, Huffman coding runs in

$$\mathcal{O}(m) + \mathcal{O}(n \log n) + \mathcal{O}(n \log n) + \mathcal{O}(n) = \mathcal{O}(m + n \log n)$$